



Section 16 / 24

Go to Questions ▾

Using the Stack

We have so far learned two types of Control Instructions: `Loops` and `Branching`. Before we discuss `Functions`, we need to understand how to use the memory `Stack`. In the `Computer Architecture` section, we discussed how the RAM is segmented into four different segments, and each application is allocated its Virtual Memory and its segments. We have also discussed the `text` segment where the application's assembly instructions are loaded into for the CPU to access and the `data` segment that holds the application's variables. So, now let's start discussing the `Stack`.

The Stack

The stack is a segment of memory allocated for the program to store data in, and it is usually used to store data and then retrieve them back temporarily. The top of the stack is referred to by the Top Stack Pointer `rsp`, while the bottom is referred to by the Base Stack Pointer `rbp`.

We can `push` data into the stack, and it will be at the top of the stack (i.e. `rsp`), and then we can `pop` data out of the stack into a register or a memory address, and it will be removed from the top of the stack.



Instruction	Description	Example
<code>push</code>	Copies the specified register/address to the top of the stack	<code>push rax</code>
<code>pop</code>	Moves the item at the top of the stack to the specified register/address	<code>pop rax</code>

The stack has a `Last-in First-out` (`LIFO`) design, which means we can only `pop` out the last element `push` ed into the stack. For example, if we `push rax` into the stack, the top of the stack would now be the value of `rax` we just pushed. If we `push` anything on top of it, we would have to `pop` them out of the stack until that value of `rax` reaches the top of the stack, then we can `pop` that value back to `rax` .

The following table demonstrates how the stack works:

We can click on `push` to push a value from `rax` to the stack, and you can click on `pop` to pop the top value from the stack into `rax` .

0xabcdef	<-- Top of Stack (<code>\$rsp</code>)

0x1234567890

<-- Bottom of Stack
(`$rbp`)

`rax:`

push

pop

Reset
Stack

Usage With Functions/Syscalls

We will primarily be pushing data from registers into the stack before we call a `function` or call a `syscall`, and then restore them after the function and the syscall. This is because `functions` and `syscalls` usually use the registers for their processing, and so if the values stored in the registers will get changed after a function call or a syscall, we will lose them.

For example, if we wanted to call a syscall to print `Hello World` to the screen and retain the current value stored in `rax`, we would `push rax` into the stack. Then we can execute the syscall and afterward `pop` the value back to `rax`. So, this way, we would be able to both execute the syscall and retain the value of `rax`.

PUSH/POP

Our code currently looks like the following:

```

global _start

section .text
_start:
    xor rax, rax    ; initialize rax to 0
    xor rbx, rbx    ; initialize rbx to 0
    inc rbx         ; increment rbx to 1
loopFib:
    add rax, rbx    ; get the next number
    xchg rax, rbx   ; swap values
    cmp rbx, 10     ; do rbx - 10
    js loopFib      ; jump if result is <0

```

Let's assume that we want to call a `function` or a `syscall` before entering the loop. To preserve our registers, we will need to `push` to the stack all of the registers we are using and then pop them back after the `syscall`.

To `push` value into the stack, we can use its name as the operand, as in `push rax`, and the value will be `copied` to the top of the stack. When we want to retrieve that value, we first need to be sure that it is on the top of the stack, and then we can specify the storage location as the operand, as in `pop rax`, after which the value will be `moved` to `rax`, and will be `removed` from the top of the stack. The value below it will now be on top of the stack (as shown in the exercise above).

Since the stack has a **LIFO** design, when we restore our registers, we have to do them in **reverse order**. For example, if we **push rax** and then **push rbx**, when we restore, we have to **pop rbx** and then **pop rax**.

So, to save our registers before entering the loop, let's push them to the stack. Luckily, we are only using `rax` and `rbx`, and so we will only need to `push` these two registers to the stack and then `pop` them after the syscall, as follows:

```
nasm

global _start

section .text
_start:
    xor rax, rax    ; initialize rax to 0
    xor rbx, rbx    ; initialize rbx to 0
    inc rbx         ; increment rbx to 1
    push rax        ; push registers to stack
    push rbx
    ; call function
    pop rbx         ; restore registers from stack
    pop rax
...SNIP...
```

Note how restoring the registers with `pop` was in reverse order.

Now, let's assemble our code and test it with `gdb`:

```
$ ./assembler.sh fib.s -g
gef> b _start
gef> r
...SNIP...
gef> si
gef> si
gef> si
```

```
registers — $rax : 0x0
```

\$rbx : 0x1

```
stack — 0x00007fffffff410 | +0x0000: 0x0000000000000001 ←
$rsp0x00007fffffff418 | +0x0008: 0x0000000000000000
0x00007fffffff420 | +0x0010: 0x0000000000000000
0x00007fffffff428 | +0x0018: 0x0000000000000000
0x00007fffffff430 | +0x0020: 0x0000000000000000
0x00007fffffff438 | +0x0028: 0x0000000000000000
0x00007fffffff440 | +0x0030: 0x0000000000000000
0x00007fffffff448 | +0x0038: 0x0000000000000000
```

code:x86:64 —

```
→ 0x40100e <_start+9>      push    rax
    0x40100f <_start+10>     push    rbx
    0x401010 <_start+11>     pop     rbx
    0x401011 <_start+12>     pop     rax
```

We see that before we execute `push rax`, we have `rax = 0x0` and `rbx = 0x1`. Now let's `push` both `rax` and `rbx`, and see how the stack and the registers change:

```
registers — $rax : 0x0
$rbx      : 0x1
```

```
stack — 0x00007fffffff408 | +0x0000: 0x0000000000000000 ←
$rsp0x00007fffffff410 | +0x0008: 0x0000000000000001
0x00007fffffff418 | +0x0010: 0x0000000000000000
0x00007fffffff420 | +0x0018: 0x0000000000000000
0x00007fffffff428 | +0x0020: 0x0000000000000000
0x00007fffffff430 | +0x0028: 0x0000000000000000
0x00007fffffff438 | +0x0030: 0x0000000000000000
0x00007fffffff440 | +0x0038: 0x0000000000000000
```

code:x86:64 —

```
0x40100e <loopFib+9>      push    rax
```

```

→ 0x40100f <_start+10>    push    rbx
    0x401010 <_start+11>    pop     rbx
    0x401011 <_start+12>    pop     rax

```

...SNIP...

```

registers — $rax : 0x0
$rbx      : 0x1

```

```

stack — 0x00007fffffffe400|+0x0000: 0x0000000000000001 ←
$rsp0x00007fffffffe408|+0x0008: 0x0000000000000000
0x00007fffffffe410|+0x0010: 0x0000000000000001
0x00007fffffffe418|+0x0018: 0x0000000000000000
0x00007fffffffe420|+0x0020: 0x0000000000000000
0x00007fffffffe428|+0x0028: 0x0000000000000000
0x00007fffffffe430|+0x0030: 0x0000000000000000
0x00007fffffffe438|+0x0038: 0x0000000000000000

```

```

code:x86:64 —
    0x40100e <_start+9>    push    rax
    0x40100f <_start+10>   push    rbx
→   0x401010 <_start+11>   pop     rbx
    0x401011 <_start+12>   pop     rax

```

We see that after we **push** ed both **rax** and **rbx** , we have the following values on the top of our stack:

```

0x00007fffffffe408|+0x0000: 0x0000000000000001 ← $rsp
0x00007fffffffe410|+0x0008: 0x0000000000000000

```

shellsession

We see that at the top of the stack, we have the last value we pushed, which is **rbx = 0x1** , and just below it, we have the value we pushed before it **rax = 0x0** . This is as we expected and

similar to the stack exercise above. We also notice that after we pushed our values, they remained in the registers, meaning a push is, in fact, a copy to stack.

Now let's assume that we finished executing a `print` function, and want to retrieve our values back, so we continue with the `pop` instructions:

```
registers — $rax : 0x0
$rbx      : 0x1

stack — 0x00007fffffffef408 | +0x0000: 0x0000000000000000 ←
$rsp0x00007fffffffef410 | +0x0008: 0x0000000000000001
0x00007fffffffef418 | +0x0010: 0x0000000000000000
0x00007fffffffef420 | +0x0018: 0x0000000000000000
0x00007fffffffef428 | +0x0020: 0x0000000000000000
0x00007fffffffef430 | +0x0028: 0x0000000000000000
0x00007fffffffef438 | +0x0030: 0x0000000000000000
0x00007fffffffef440 | +0x0038: 0x0000000000000000

code:x86:64 —
    0x40100e <_start+9>    push    rax
    0x40100f <_start+10>   push    rbx
    0x401010 <_start+11>   pop     rbx
→  0x401011 <_start+12>   pop     rax
rax
...SNIP...

registers — $rax : 0x0
$rbx      : 0x1

stack — 0x00007fffffffef410 | +0x0000: 0x0000000000000001 ←
$rsp0x00007fffffffef418 | +0x0008: 0x0000000000000000
0x00007fffffffef420 | +0x0010: 0x0000000000000000
0x00007fffffffef428 | +0x0018: 0x0000000000000000
0x00007fffffffef430 | +0x0020: 0x0000000000000000
0x00007fffffffef438 | +0x0028: 0x0000000000000000
```

```
0x00007fffffffef440 | +0x0030: 0x0000000000000000  
0x00007fffffffef448 | +0x0038: 0x0000000000000000
```

```
code:x86:64 ———  
    0x40100f <_start+9>      push    rax  
    0x40100f <_start+10>     push    rbx  
    0x401010 <_start+11>     pop     rbx  
    0x401011 <_start+12>     pop     rax  
→   0x401011 <loopFib+0>     add     rax,  
rbx
```

We see that after `pop`ing two values from the top of the stack, they were removed from the stack, and the stack now looks exactly as when we first started. Both values were placed back in `rbx` and `rax`. We may not have seen any difference since they were not changed in the registers in this case.

Using the stack is very simple. The only thing we should keep in mind is the order we push our registers and the state of the stack to safely restore our data and not restore a different value by `pop` when a different value is at the top of the stack.

We can remove the `push` and `pop` instructions from our code for now, and we will use them when we get into function calls. With that, we should be ready to use `syscall` and `function` calls. Let us discuss `syscalls` next.

Connect to HTB



Pwnbox

Your own web-based Parrot Linux instance to play our labs.



Switching Pwnbox location will terminate the spawned Pwnbox.

Pwnbox Location

UK



Start Pwnbox

∞ spawns left

• Offline



Enable step-by-step solutions



PRO

⚠ Answer questions in English to ensure an accurate response.

Question 1

 +1

 +40



Debug the attached binary to find the flag being pushed to the stack

Write your answer

 Submit



Hint



Download File

← Previous

Section 16 / 24

 +20

Next

Mark Complete & Next